INTELLIGENT EMAIL MANAGEMENT SYSTEM

A Project Based Learning Report submitted in partial fulfilment of the requirements for the award of the degree

of

Bachelor of Technology

in the Department of Artificial Intelligence and Data Science

Course: DSE&DBD | Course Code: 25CS1302E

Submitted by

2510080010— Y. Harshitha
2510080011 — T. Srujan Kumar Reddy
2510080012— M. Sai Vignesh
2510080026-Chavali Mary Siri Vennela

Project Track: Database Management Systems (DBMS)
Repository: Intelligent_Email_Management_Final
Stack: PostgreSQL, FastAPI, React + Vite, scikit-learn

Under the guidance of

Dr. Laiphangbam Melinda



Department of Department of Artificial Intelligence and Data Science

Koneru Lakshmaiah Education Foundation, Aziz Nagar

Aziz Nagar – 500075

OCTOBER – 2026

Abstract

This project presents an intelligent email management system developed as a DBMS-focused Project Based Learning application. The system uses PostgreSQL as the core relational data store, with a FastAPI service and React web client providing the application layer. Its main functions include authenticated access, inbox and folder management, message composition and status changes, category management, keyword and multi-criteria search, dashboard reporting, audit logging, and database-side automation. The database is organized into five core tables—users, categories, emails, spam_detection, and email_audit_log—with primary and foreign keys, constraints, indexes, stored functions, and PostgreSQL triggers. A machine-learning spam classifier is included as a supporting feature: subject and body text are transformed with TF–IDF using unigrams and bigrams, and Multinomial Naive Bayes is selected from two candidate models using F1 score. The supplied dataset contains 179 labeled examples, and the recorded 36-message holdout results are 94.44% accuracy, 90.00% precision, 100.00% recall, and 94.74% F1. When a stored message is classified as spam, a PostgreSQL trigger can update its status and category and create an audit record. The project is an academic prototype with manually entered or API-submitted email data; the source archive contains no live Gmail, Outlook, IMAP, or other external mail-provider integration.

List of Figures

Figure 1. Implemented multi-tier architecture and message-classification dataflow.

Figure 2. Email management interface and message workflow output.

Figure 3. PostgreSQL database, audit log, and trigger output.

Figure 4. Spam detection and automatic categorization output.

Figure 5. Dashboard and reporting output.

List of Tables

Table 1. Spam-classifier performance on the holdout set.

Table 2. Confusion matrix for the holdout set.

Table 3. Core relational entities and their responsibilities.

Table of Contents

Abstract
List of Figures
List of Tables
Introduction
Methodology
System Architecture and Workflow
Relational Database Design
Dataset Preparation and Text Representation
Model Training and Inference
Application Interface, API, and Verification
Experiments
Experimental Setup
Evaluation Measures and Functional Verification
Results
Conclusion and Future Work
References
Database Functions, Triggers, and Demonstration Records
Implemented API Surface

INTELLIGENT EMAIL MANAGEMENT SYSTEM

1. INTRODUCTION

Email is a common channel for academic, personal, and organizational communication, but an email application also has to manage users, messages, folders, categories, search, status changes, and history in a consistent way. The central challenge addressed by this project is therefore a database-oriented one: how to store email information in a normalized relational structure and support reliable retrieval, updates, classification, and auditability through an application interface. The system models each message with a sender, receiver, subject, body, received time, status, and optional category. User accounts and categories are maintained separately and are connected to messages through foreign keys. A separate spam_detection table stores the classification result, score, explanation, and detection time, while email_audit_log records important message events. This separation reduces redundancy and allows database operations to remain clear and maintainable.

The project also includes a machine-learning spam classifier as a supporting intelligent feature. Its purpose is not to replace the DBMS layer, but to generate a classification that can be persisted and acted upon by the database. When a stored email is classified as Spam, a PostgreSQL trigger can update the message status, assign the Spam category, and create an audit event. The application therefore demonstrates how an intelligent component can feed a relational workflow.

The project objectives are: (1) design a normalized relational database for users, emails, categories, spam detections, and audit records; (2) implement CRUD operations, search, filtering, and reporting through a REST API; (3) demonstrate primary keys, foreign keys, constraints, indexes, stored functions, and triggers; (4) provide an application interface for practical database interaction; and (5) integrate a reproducible spam-classification module that stores its results in the database.

The scope is limited to the supplied academic implementation. Email records are entered through the application interface or API and are stored in a local PostgreSQL database. The source archive contains seeded demonstration data and does not include a connector to Gmail, Outlook, IMAP, or another external mailbox service. The included ML dataset is small and project-specific, so its

reported performance is a prototype benchmark rather than evidence of production spam-filtering accuracy.

The remainder of this report presents the system architecture, relational design, database functions and triggers, application workflow, supporting spam-classification method, verification approach, and recorded results. The final section identifies practical improvements for security, testing, database deployment, model evaluation, and future external-mail integration.

2.METHODOLOGY

a. System Architecture and Workflow

The system follows a layered architecture in which PostgreSQL is the central persistence and DBMS component. A React + Vite client provides the user interface, while a Python FastAPI service exposes REST endpoints and uses SQLAlchemy for database access. The workflow is: authenticate; create or browse email records; view, search, filter, or update messages; run spam detection when required; and review dashboard, audit, and report information. Database triggers and stored functions provide automation and database-side processing, while the ML module acts as a supporting classification service.

For a new email, the application accepts the receiver, subject, and body from the user interface and sends the data to the backend. The backend stores the message in the emails table. A prediction can then be requested through POST /api/spam/predict for a non-persistent check or POST /api/spam/detect/{email_id} for a stored message. In the latter workflow, the classification is written to spam_detection; the PostgreSQL trigger then synchronizes the email status/category and records the automated action in email_audit_log. This keeps the database responsible for the message state and audit trail.

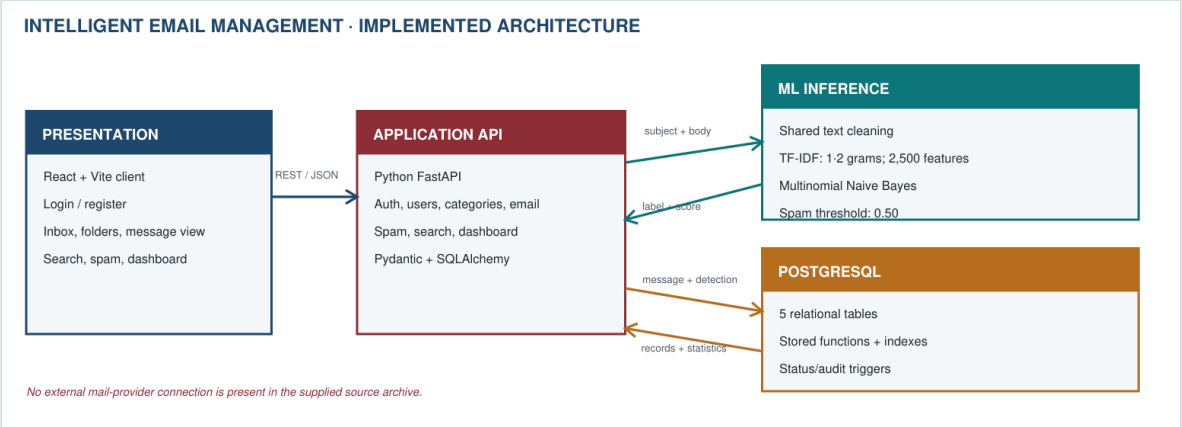


Figure 1. Full-stack architecture and operational dataflow implemented in the supplied source archive.

b. Relational Database Design

The schema defines five core tables: users, categories, emails, spam_detection, and email_audit_log. The emails table references users through sender_id and receiver_id and references categories through category_id. spam_detection has a one-to-one relationship with emails through a unique email_id. Primary keys, foreign keys, unique constraints, check constraints, and indexed columns maintain integrity and support common queries. A PostgreSQL GIN index over subject and body supports full-text search. The design is normalized to 3NF so user and category details are not unnecessarily repeated in every email row.

c. Database Functions, Triggers, and Demonstration Records

The SQL scripts implement database-side functionality through fn_get_user_email_stats(user_id), fn_get_system_dashboard_stats(), and fn_search_emails(...). The trg_audit_email trigger records CREATED and STATUS_CHANGED events for email rows. The trg_spam_detection_sync trigger calls the spam synchronization function; when a detection is classified as Spam, it updates emails.status, assigns the Spam category when available, and writes an AUTO_SPAM_FLAGGED audit record with the model score. The seed script creates five categories, six demonstration users, eleven sample emails, and eleven initial detection records. These records are controlled fixtures for demonstrating the DBMS workflow, not live email-provider data.

Table 3. Core relational entities implemented in database/schema.sql.

Entity	Key fields and relationships	Purpose
users	user_id; unique email; password_hash	Account identity and authentication
categories	category_id; unique category_name	System and custom message categories

emails	email_id; sender_id/receiver_id → users; category_id → categories	Message content, participants, status, and received time
spam_detection	detection_id; unique email_id → emails; score, label, reason	Persisted classifier output and explanation
email_audit_log	audit_id; email_id; action, previous/new status, details	Record message creation and status/spam events

d. Dataset Preparation and Text Representation

The supporting classifier combines the subject and body of each email and applies the same normalization during training and inference. Text is lowercased; URLs, email addresses, and dollar amounts are mapped to stable tokens; punctuation is removed; and repeated whitespace is collapsed. TfidfVectorizer uses up to 2,500 features, unigrams and bigrams, English stop-word removal, and sublinear term frequency. The dataset contains 179 labeled records and is split with stratification and random_state=42. Because the dataset is small, these measurements are treated as an initial project benchmark.

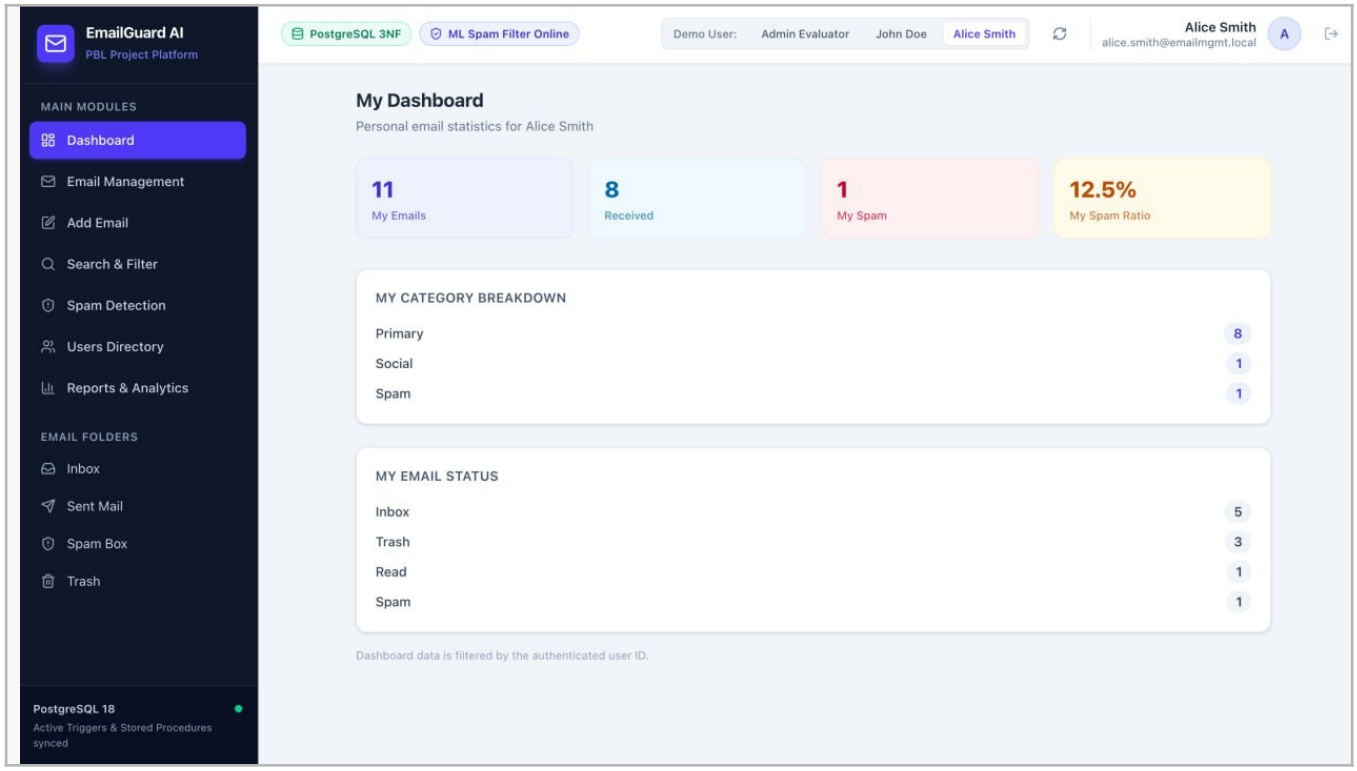
e. Model Training and Inference

Two supervised models are trained as candidates: Multinomial Naive Bayes with alpha=0.5 and Logistic Regression with C=2.0 and a maximum of 1,000 iterations. The implementation selects the candidate with the larger F1 score on the holdout partition and saves the estimator, vectorizer, and evaluation metrics as artifacts. During inference, the same subject/body representation is used and a 0.50 decision threshold produces the Spam/Not Spam label. For a stored email, the result is persisted in spam_detection so that PostgreSQL automation can update the corresponding message state.

f. Application Interface, API, and Verification

The React client provides dashboard, inbox/folder, message detail, search, spam detection, users, and reports views. The FastAPI service groups endpoints for authentication, users, categories, email operations, spam detection, search, dashboard statistics, reports, and health checks. Passwords are hashed with bcrypt and successful authentication returns a signed JWT. SQLAlchemy models and Pydantic schemas mediate database access and API validation. The source contains 20 pytest test functions covering authentication, database schema/functions/triggers, dashboard/search, email operations, and spam prediction. These test definitions show intended verification coverage; the archive does not include a test-execution report.

Figure 2. Output screenshot — email management interface.



g. Implemented API Surface

The main route groups are authentication—POST /api/auth/register, POST /api/auth/login, and GET /api/auth/me; users—GET /api/users and GET /api/users/{user_id}; categories—GET and POST /api/categories; and email management—GET /api/emails, GET and POST email operations, PUT /api/emails/{email_id}/status, and DELETE /api/emails/{email_id}. Deletion first moves a message to Trash and permanently deletes it if it is already in Trash. These endpoints provide the application-level interface through which the PostgreSQL database is demonstrated.

Spam routes are POST /api/spam/predict, POST /api/spam/detect/{email_id}, and GET /api/spam/detections. GET /api/search accepts keyword, sender, receiver, category_id, spam_status, and status filters. Dashboard routes expose /api/dashboard/stats, /api/dashboard/user-stats, and /api/dashboard/reports; GET /api/health reports API and database status. The React navigation includes Dashboard, Email Management, Add Email, Search & Filter, Spam Detection, Users Directory, Reports & Analytics, Inbox, Sent Mail, Spam Box, and Trash.

h. Database Automation and Auditability

An AFTER INSERT OR UPDATE trigger on emails writes CREATED and STATUS_CHANGED events to the audit table. A second trigger on spam_detection responds when the classification is Spam by updating the email state and category and writing an AUTO_SPAM_FLAGGED record with the model score. These mechanisms demonstrate database

automation and auditability without requiring the frontend to manually maintain every audit entry. For deployment beyond a controlled academic demonstration, the wildcard CORS policy, development fallback signing key, and endpoint authorization rules should be reviewed and restricted.

Figure 3. Output screenshot — PostgreSQL tables, trigger result, or audit log.

	email_id [PK] integ	sender_id integer	receiver_id integer	subject character varying (255)	body text	received_at timestamp with	status characte	category_ integer
1	1	4	2	PBL Project Review 4 Demonstration Sched...	Dear John, Please be ready with your project dem...	2026-10-02...	Inbox	
2	2	3	2	Database Schema Normalization Discussio...	Hey John, I reviewed the 3NF schema for the ema...	2026-10-03...	Read	
3	3	1	2	Department Seminar on Distributed Machin...	All students are invited to attend the guest lectur...	2026-10-04...	Inbox	
4	4	2	4	Submitted Review 4 Presentation Slides an...	Dear Professor Wilson, Attached are the final pre...	2026-10-04...	Inbox	
5	5	3	2	Team lunch after project presentation	Hey John, everyone in the team is heading out to ...	2026-10-05...	Inbox	
6	6	4	3	Guidelines for final thesis submission	Dear Alice, ensure you format the bibliography fol...	2026-10-05...	Inbox	
7	7	5	2	CONGRATULATIONS! You Won $1,000,000 ...	Dear Lucky Winner, Your email account has been ...	2026-09-30...	Spam	
8	8	5	2	URGENT: Your Bank Account Access Is Sus...	Security Alert: We detected unauthorized login att...	2026-10-01...	Spam	
9	9	6	2	Exclusive Offer: Cheap Meds, Weight Loss ...	Order generic pharmaceuticals online without pre...	2026-10-02...	Spam	
10	10	5	3	Claim Your Unclaimed Inheritance of $5.5M...	Dear Sir/Madam, I represent an estate with $5,50...	2026-10-03...	Spam	
11	11	6	1	Double Your Crypto Investment in 48 Hours...	Automated AI trading bot guarantees 300% weekl...	2026-10-04...	Trash	
12	14	2	3	Automated Test Email via API	Hi Alice, please confirm receipt of the automated ...	2026-10-05...	Inbox	
13	15	2	3	Status Change Test	Checking status change trigger logging.	2026-10-05...	Inbox	
14	16	2	2	Exclusive Discount on Replica Luxury Watc...	Buy cheap fake Rolex watches 90% off with free o...	2026-10-05...	Inbox	
15	19	2	3	Automated Test Email via API	Hi Alice, please confirm receipt of the automated ...	2026-10-05...	Trash	
16	24	2	3	Automated Test Email via API	Hi Alice, please confirm receipt of the automated ...	2026-10-05...	Trash	
17	27	1	2	Live system verification test	This is a live API test confirming email compose ...	2026-10-05...	Inbox	
18	30	2	3	Automated Test Email via API	Hi Alice, please confirm receipt of the automated ...	2026-10-05...	Trash	
19	33	3	3	DBMS Report Submission	This is to inform you that the deadline for the DB...	2026-10-06...	Inbox	

2. EXPERIMENTS

A. *Experimental Setup*

The shipped dataset contains 179 labeled records with three fields: label, subject, and body. It is nearly balanced, with 90 legitimate (ham) and 89 spam examples. The training script combines subject and body, cleans the text, and uses a stratified 80/20 split with random_state=42. The recorded evaluation artifact contains 143 training examples and 36 holdout examples. Multinomial Naive Bayes and Logistic Regression are compared, with Multinomial Naive Bayes selected on the recorded holdout F1. Alongside this ML experiment, the DBMS implementation provides five core tables, stored functions, indexes, triggers, seed data, and a defined pytest verification suite.

a. Evaluation Measures and Functional Verification

Accuracy is the proportion of correct holdout predictions. Precision measures how many messages predicted as spam are actually spam; recall measures how many actual spam messages are detected; and F1 is the harmonic mean of precision and recall. For the DBMS portion, functional verification focuses on API behavior, relational objects, stored functions, triggers, audit logging, search, and

automatic spam categorization. The source contains 20 test functions, but no executed test output is included. Model selection and evaluation also use the same small holdout partition, so the reported ML score should be treated as preliminary.

The functional test definitions cover registration/login/profile, email listing and composition, status updates, spam prediction and persistence, dashboard/search/report responses, table presence, stored functions, audit logging, and automatic spam categorization. The database therefore forms the main verification target, while the classifier is tested as one supporting service. Reproducibility requires the configured PostgreSQL database and application dependencies described in the source package.

3. RESULTS

The main implemented result is a working DBMS-centered email-management workflow: five normalized relational tables store users, categories, emails, spam detections, and audit events; indexes support common lookups and full-text search; stored functions provide statistics and search; and triggers automate audit logging and spam categorization. The supplied model-evaluation artifact additionally records 94.44% accuracy, 90.00% precision, 100.00% recall, and 94.74% F1 on a 36-message holdout set. These ML values are useful for demonstrating integration, but they should not be interpreted as production spam-filtering performance because the dataset is small and project-specific.

Table 1. Model evaluation on the stratified holdout set (n = 36).

Measure	Recorded value	Interpretation
Accuracy	0.9444 (94.44%)	34 of 36 predictions correct
Precision	0.9000 (90.00%)	18 of 20 spam predictions correct
Recall	1.0000 (100.00%)	18 of 18 spam messages detected
F1 score	0.9474 (94.74%)	Harmonic mean of precision and recall

Holdout size	36 messages	18 ham and 18 spam

Table 2. Confusion matrix for the holdout set (rows = actual class; columns = predicted class).

Actual class	Predicted Ham	Predicted Spam
Ham (legitimate)	16	2
Spam	0	18

The model artifact identifies Multinomial Naive Bayes (alpha=0.5) as the selected estimator and records 2,500 vectorizer features. Its recall means that no spam example in this particular test split was missed, while the two false positives show that legitimate mail can be misclassified. From the DBMS perspective, the important result is that the prediction can be persisted in spam_detection and used by a PostgreSQL trigger to update the email record and create an audit event.

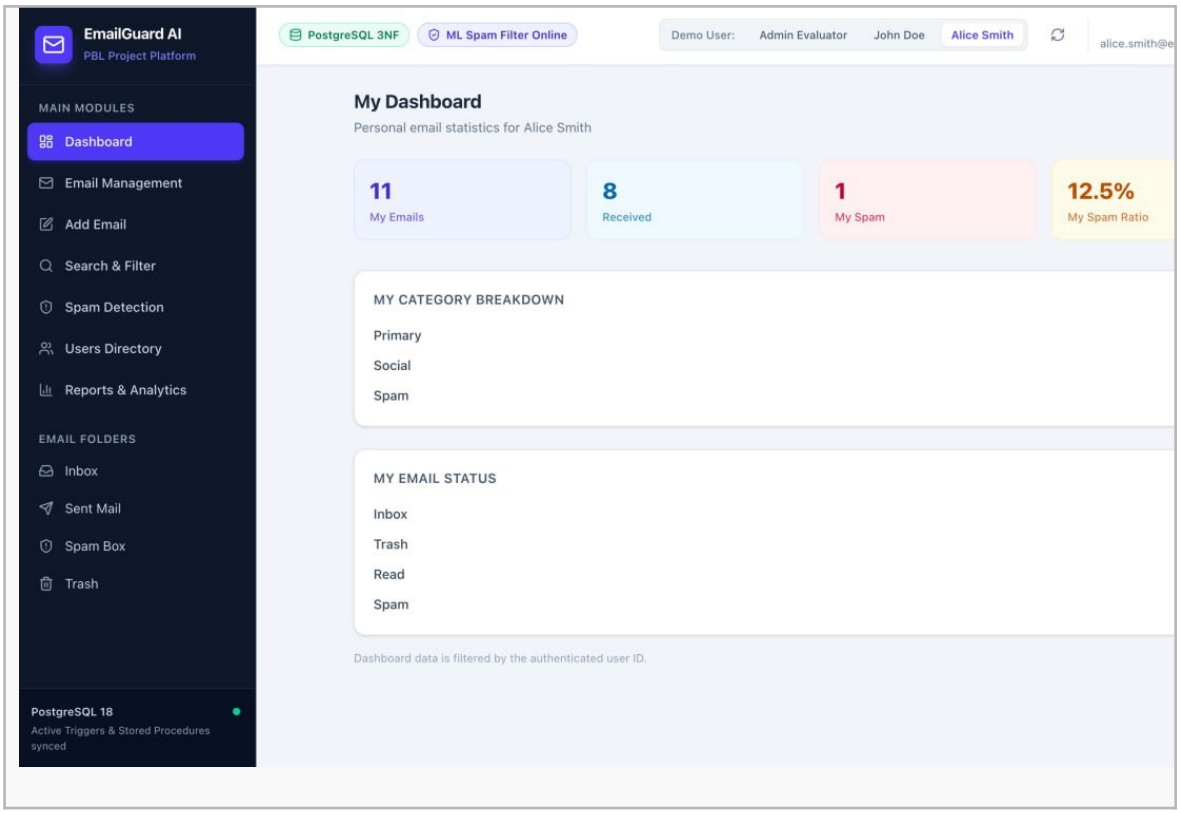
Figure 4. PostgreSQL screenshot.

	audit_id [PK] integ	email_id integer	action character varying (50)	previous_ characte	new_statu character	details text	changed_at timestamp with
1	1	1	CREATED	[null]	Inbox	Email created and delivered to receiver inbox	2026-10-05 ...
2	2	2	CREATED	[null]	Read	Email created and delivered to receiver inbox	2026-10-05 ...
3	3	3	CREATED	[null]	Inbox	Email created and delivered to receiver inbox	2026-10-05 ...
4	4	4	CREATED	[null]	Inbox	Email created and delivered to receiver inbox	2026-10-05 ...
5	5	5	CREATED	[null]	Inbox	Email created and delivered to receiver inbox	2026-10-05 ...
6	6	6	CREATED	[null]	Inbox	Email created and delivered to receiver inbox	2026-10-05 ...
7	7	7	CREATED	[null]	Inbox	Email created and delivered to receiver inbox	2026-10-05 ...
8	8	8	CREATED	[null]	Inbox	Email created and delivered to receiver inbox	2026-10-05 ...
9	9	9	CREATED	[null]	Inbox	Email created and delivered to receiver inbox	2026-10-05 ...
10	10	10	CREATED	[null]	Inbox	Email created and delivered to receiver inbox	2026-10-05 ...
11	11	11	CREATED	[null]	Inbox	Email created and delivered to receiver inbox	2026-10-05 ...
12	12	7	STATUS_CHANGED	Inbox	Spam	Status updated from Inbox to Spam	2026-10-05 ...
13	13	7	AUTO_SPAM_FLAGGED	Inbox	Spam	Automated ML Spam Classifier flagged message with confidence 98.0%	2026-10-05 ...
14	14	8	STATUS_CHANGED	Inbox	Spam	Status updated from Inbox to Spam	2026-10-05 ...
15	15	8	AUTO_SPAM_FLAGGED	Inbox	Spam	Automated ML Spam Classifier flagged message with confidence 96.0%	2026-10-05 ...
16	16	9	STATUS_CHANGED	Inbox	Spam	Status updated from Inbox to Spam	2026-10-05 ...
17	17	9	AUTO_SPAM_FLAGGED	Inbox	Spam	Automated ML Spam Classifier flagged message with confidence 92.0%	2026-10-05 ...
18	18	10	STATUS_CHANGED	Inbox	Spam	Status updated from Inbox to Spam	2026-10-05 ...
19	19	10	AUTO_SPAM_FLAGGED	Inbox	Spam	Automated ML Spam Classifier flagged message with confidence 97.0%	2026-10-05 ...
20	20	11	STATUS_CHANGED	Inbox	Spam	Status updated from Inbox to Spam	2026-10-05 ...
21	21	11	AUTO_SPAM_FLAGGED	Inbox	Spam	Automated ML Spam Classifier flagged message with confidence 94.0%	2026-10-05 ...
22	22	12	CREATED	[null]	Inbox	Email created and delivered to receiver inbox	2026-10-05 ...
23	23	12	STATUS_CHANGED	Inbox	Read	Status updated from Inbox to Read	2026-10-05 ...
24	24	13	CREATED	[null]	Inbox	Email created and delivered to receiver inbox	2026-10-05 ...
25	25	13	STATUS_CHANGED	Inbox	Spam	Status updated from Inbox to Spam	2026-10-05 ...

The source package also includes dashboard and persistence code, database triggers, and a 20-test pytest suite. These artifacts demonstrate planned integration and verification coverage. No executed test results or deployment observations are supplied, so this report does not claim that the complete application was verified in a production environment.

Figure 5. Output screenshot — dashboard and reporting.





4. CONCLUSION AND FUTURE WORK

The project implements an integrated email-management system with PostgreSQL at its core. The relational design separates users, categories, emails, spam detections, and audit records; foreign keys and constraints preserve integrity; indexes support retrieval; stored functions provide database-side statistics and search; and triggers automate audit logging and spam categorization. The React and FastAPI layers provide a practical interface to these DBMS capabilities. The ML classifier is an additional feature that supplies spam labels and scores which can be stored and acted upon by the database.

Future work should strengthen the database deployment and verification process by running the full pytest suite against a reproducible PostgreSQL environment, adding more representative seed and test cases, documenting query plans for important searches, and measuring index performance on a larger dataset. For the ML component, the dataset should be expanded, duplicates removed across partitions, and an untouched final test set retained. Cross-validation, calibration, and time-based evaluation would provide a stronger estimate of generalization.

Engineering improvements include stricter authorization for every endpoint, environment-based secrets, restricted CORS, database backup and migration procedures, and clearer transaction/error handling. User feedback could be used to improve spam classification over time. A live email-

provider integration is outside the current implementation; if added later, it should use user-authorized OAuth, least-privilege scopes, privacy safeguards, and explicit consent for message access.

REFERENCES

[1] [1] A. McCallum and K. Nigam, “A comparison of event models for Naive Bayes text classification,” in Proceedings of the AAAI-98 Workshop on Learning for Text Categorization, 1998, pp. 41–48.

[2] [2] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988.

[3] [3] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[4] PostgreSQL Global Development Group, “Full Text Search,” PostgreSQL Documentation. [Online]. Available: https://www.postgresql.org/docs/current/textsearch.html. Accessed: 06-Oct-2026.

[4] [5] SQLAlchemy authors, “ORM Quick Start,” SQLAlchemy 2.0 Documentation. [Online]. Available: https://docs.sqlalchemy.org/en/20/orm/quickstart.html. Accessed: 06-Oct-2026.

[5] [6] Intelligent Email Management System, supplied project source archive: PostgreSQL schema, functions, triggers, application source, ML training/evaluation scripts, and test definitions, 2026.